

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_RDONLY);
    char ch;

    off_t size = lseek(fd, 0, SEEK_END);

    for (int i = size - 1; i >= 0; i--) {
        lseek(fd, i, SEEK_SET);
        read(fd, &ch, 1);
        write(1, &ch, 1);
    }

    close(fd);
    return 0;
}
//create a file called file.txt and write the following text:
//Hello, World!
//then cc <q1a.c>
//./a.out
//you should see the text printed in reverse order.
```

```
#include <stdio.h>
#include <sys/stat.h>

int main() {
    struct stat s;

    stat("file.txt", &s);

    printf("File Size: %ld bytes\n", s.st_size);
    printf("Inode Number: %ld\n", s.st_ino);
    printf("Number of Links: %ld\n", s.st_nlink);
    printf("User ID: %d\n", s.st_uid);
    printf("Group ID: %d\n", s.st_gid);
    printf("Block Size: %ld\n", s.st_blksize);

    return 0;
}
//create a file called file.txt and write the following text:
//Hello, World!
//then cc <q2a.c>
//./a.out
//you should see the file size, inode number, number of links, user
id, group id, and block size.
```

```
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    struct dirent *d;
    struct stat s;
    DIR *dir = opendir(".");

    while ((d = readdir(dir)) != NULL) {
        stat(d->d_name, &s);

        if (S_ISREG(s.st_mode) && s.st_size == 0) {
            remove(d->d_name);
            printf("Deleted: %s\n", d->d_name);
        }
    }

    closedir(dir);
    return 0;
}
//then cc <3a.c> create one empty file called empty.txt
//./a.out
//you should see the empty.txt file deleted.
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // create hard link
    link("file.txt", "hardlink.txt");

    // create soft link
    symlink("file.txt", "softlink.txt");

    printf("Hard link and Soft link created\n");

    return 0;
}
//then cc <4a.c>
//./a.out
//you should see the hard link and soft link created.
```

```
#include <stdio.h>
#include <sys/stat.h>
#include <utime.h>

int main() {
    struct stat s;
    struct utimbuf t;

    // get time of source file
    stat("file.txt", &s);

    // assign times
    t.actime = s.st_atime;    // access time
    t.modtime = s.st_mtime;  // modification time

    // set time to destination file
    utime("file2.txt", &t);

    printf("Time copied successfully\n");

    return 0;
}
//then cc <5a.c>
//./a.out file.txt file2.txt
//you should see the time copied successfully.
```

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_RDWR | O_APPEND);
    char buf[100];
    int n = 5; // number of characters to read

    // read n characters from beginning
    lseek(fd, 0, SEEK_SET);
    read(fd, buf, n);

    // duplicate fd to STDOUT (1)
    dup2(fd, 1);

    // write back (append)
    write(1, buf, n);

    close(fd);
    return 0;
}
//then cc <6a.c>
//./a.out file.txt
//you should see the first 5 characters of the file printed to the
console.
```

```

#include <stdio.h>
#include <setjmp.h>
#include <stdlib.h>

static void f1(int, int, int, int);
static void f2(void);

static jmp_buf jb;
static int gv;    // global variable

int main() {
    int av = 2;           // automatic variable
    register int rv = 3;  // register variable
    volatile int vv = 4;  // volatile variable
    static int sv = 5;    // static variable

    gv = 1;

    if (setjmp(jb) != 0) {
        printf("After longjmp:\n");
        printf("gv=%d av=%d rv=%d vv=%d sv=%d\n",
                gv, av, rv, vv, sv);
        exit(0);
    }

    // modify values
    gv = 95;
    av = 96;
    rv = 97;
    vv = 98;
    sv = 99;

    f1(av, rv, vv, sv);

    return 0;
}

static void f1(int i, int j, int k, int l) {
    printf("In f1():\n");
    printf("gv=%d av=%d rv=%d vv=%d sv=%d\n",
            gv, i, j, k, l);

    f2();
}

static void f2(void) {
    longjmp(jb, 1);
}
//then cc <7a.c>
//./a.out
//you should see the values of a, b, and c printed to the console.

```

```

#include <stdio.h>
#include <sys/stat.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    struct stat s;

    if (argc != 2) {
        printf("Usage: %s filename\n", argv[0]);
        exit(1);
    }

    if (stat(argv[1], &s) < 0) {
        perror("stat");
        exit(1);
    }

    if (S_ISREG(s.st_mode))
        printf("Regular file\n");
    else if (S_ISDIR(s.st_mode))
        printf("Directory\n");
    else if (S_ISCHR(s.st_mode))
        printf("Character device\n");
    else if (S_ISBLK(s.st_mode))
        printf("Block device\n");
    else if (S_ISFIFO(s.st_mode))
        printf("FIFO (pipe)\n");
    else if (S_ISLNK(s.st_mode))
        printf("Symbolic link\n");
    else if (S_ISSOCK(s.st_mode))
        printf("Socket\n");
    else
        printf("Unknown file type\n");

    return 0;
}
//then cc <8a.c>
//./a.out file.txt
//you should see the file type.
//if the file is a regular file, it will print "Regular file".
//if the file is a directory, it will print "Directory".
//if the file is a character device, it will print "Character
device".
//if the file is a block device, it will print "Block device".
//if the file is a FIFO (pipe), it will print "FIFO (pipe)".
//if the file is a symbolic link, it will print "Symbolic link".
//if the file is a socket, it will print "Socket".
//if the file is an unknown file type, it will print "Unknown file
type".

```



```

#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    mode_t new_umask = 0022;
    mode_t old_umask;
    const char *file_path = "file.txt";
    mode_t new_mode = 0644;

    // Set new umask and get old umask
    old_umask = umask(new_umask);
    printf("Old umask: %03o, New umask: %03o\n", old_umask,
new_umask);

    // Create file with full permissions
    int fd = open(file_path, O_CREAT | O_WRONLY, 0777);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    close(fd);

    // Change permissions using chmod
    if (chmod(file_path, new_mode) == -1) {
        perror("chmod");
        return 1;
    }

    printf("Permissions of %s changed to %03o\n", file_path,
new_mode);

    return 0;
}
//then cc <9a.c>
//./a.out
//you should see the old umask, new umask, and the permissions of
the file changed to 0644.

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    pid_t pid;

    // Step 1: Fork
    pid = fork();
    if (pid < 0)
        exit(1);
    if (pid > 0)
        exit(0);    // parent exits

    // Step 2: Create new session
    if (setsid() < 0)
        exit(1);

    // Step 3: Change working directory
    chdir("/");

    // Step 4: Reset file permissions
    umask(0);

    // Step 5: Close standard file descriptors
    close(0);
    close(1);
    close(2);

    // (Optional) redirect to /dev/null
    open("/dev/null", O_RDWR);
    dup(0);
    dup(0);

    // Daemon work (example loop)
    while (1) {
        // Your background task here
        sleep(5);
    }

    return 0;
}

//vi program_name.c
//cc program_name.c
//./a.out
//ps aux (question mark depicts a daemon process)
//tail -f /var/log/syslog (optional to print log statement)

```

1.

dup1.c

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd1, fd2;
    char buf[] = "abcdef";

    fd1 = open("t12.txt", O_RDWR | O_CREAT, 0644);
    if (fd1 < 0) {
        printf("Error opening file\n");
        return 1;
    }

    fd2 = dup(fd1);    // duplicate fd1

    printf("fd1 = %d, fd2 = %d\n", fd1, fd2);

    write(fd1, buf, 6);

    return 0;
}
```

2.

dup2.c

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd1, fd2;
    char buf[] = "abcdef";

    fd1 = open("t12.txt", O_RDWR | O_CREAT, 0644);
    if (fd1 < 0) {
        printf("Error opening file\n");
        return 1;
    }

    fd2 = 5;    // choose any file descriptor

    if (dup2(fd1, fd2) < 0) {    // correct usage
        printf("dup2 error\n");
        return 1;
    }

    printf("fd1 = %d, fd2 = %d\n", fd1, fd2);

    write(fd2, buf, 6);
}
```

```
    return 0;  
}  
  
//  
compile both and run both separately
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid, pid2;

    if ((pid = fork()) < 0) {
        printf("fork error\n");
    }
    else if (pid == 0) {    // child process
        printf("Child (PID=%d) exiting\n", getpid());
        exit(0);
    }
    else {
        printf("Parent (PID=%d) sleeping\n", getpid());
        sleep(5);    // child becomes zombie here

        // Solution: collect child
        if ((pid2 = waitpid(pid, NULL, 0)) != pid)
            printf("waitpid error\n");

        printf("Parent collected child PID = %d\n", pid2);
    }

    return 0;
}

//then cc <12a.c>
//./a.out
//you should see the parent and child PIDs and the child exiting.

```

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_RDONLY);
    char ch;

    if (fork() == 0) {    // child
        read(fd, &ch, 1);
        printf("Child read: %c\n", ch);
    } else {              // parent
        sleep(1); // ensure child runs first
        read(fd, &ch, 1);
        printf("Parent read: %c\n", ch);
    }

    return 0;
}

//create a file called file.txt and write the following text:
//Hello, World!
//then cc <q1b.c>
//./a.out
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int mysystem(char *cmd) {
    int pid = fork();

    if (pid == 0) { // child
        execl("/bin/sh", "sh", "-c", cmd, NULL);
    } else { // parent
        wait(NULL);
    }
    return 0;
}

int main() {
    mysystem("ls");
    return 0;
}
//then cc <2b.c>
//./a.out
//you should see the output of the ls command.
```

```

#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>

int main() {
    struct dirent *d;
    struct stat s;
    DIR *dir = opendir(".");

    while ((d = readdir(dir)) != NULL) {
        stat(d->d_name, &s);

        printf("%ld ", s.st_ino);      // inode
        printf("%ld ", s.st_nlink);    // links
        printf("%d ", s.st_uid);       // user id
        printf("%ld ", s.st_size);     // size
        printf("%s\n", d->d_name);     // file name
    }

    closedir(dir);
    return 0;
}
//then cc <3b.c>
//./a.out
//you should see the inode, links, user id, size, and file name of
each file in the current directory.

```


1.

nano echoall.c

2.

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    for (int i = 0; i < argc; i++) {  
        printf("argv[%d] = %s\n", i, argv[i]);  
    }  
    return 0;  
}
```

3.

```
cc echoall.c -o echoall
```

4.

```
nano interpreter.sh
```

5.

```
#!/bin/sh
```

```
./echoall ARG_FROM_INTERPRETER $1
```

6.

```
chmod +x interpreter.sh
```

7.

```
nano parent.c
```

8.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {  
    if (fork() == 0) {  
        execl("./interpreter.sh", "interpreter.sh", "ARG1", NULL);  
    }  
    return 0;  
}
```

9.

```
compile parent
```

10.execute

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("SIGINT caught\n");

    // reset to default action
    signal(SIGINT, SIG_DFL);
}

int main() {
    struct sigaction sa;

    sa.sa_handler = handler;
    sa.sa_flags = 0;
    sigemptyset(&sa.sa_mask);

    sigaction(SIGINT, &sa, NULL);

    while (1) {
        printf("Running... Press Ctrl+C\n");
        sleep(2);
    }

    return 0;
}

//then cc <5b.c>
//./a.out and press Ctrl+C
//you should see the SIGINT caught message.
```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_RDWR);
    struct flock f;

    // Define last 100 bytes region
    f.l_type = F_WRLCK;          // check for write lock
    f.l_whence = SEEK_END;
    f.l_start = -100;
    f.l_len = 100;

    // Check if locked
    fcntl(fd, F_GETLK, &f);

    if (f.l_type != F_UNLCK) {
        printf("Region is locked by PID: %d\n", f.l_pid);
    } else {
        printf("Region is not locked. Locking now...\n");

        // Apply lock
        f.l_type = F_WRLCK;
        fcntl(fd, F_SETLK, &f);

        // Move to last 50 bytes
        lseek(fd, -50, SEEK_END);

        char buf[51];
        read(fd, buf, 50);
        buf[50] = '\0';

        printf("Last 50 bytes: %s\n", buf);

        // Unlock
        f.l_type = F_UNLCK;
        fcntl(fd, F_SETLK, &f);

        printf("Region unlocked\n");
    }

    close(fd);
    return 0;
}

//then cc <6b.c>
//./a.out
//you should see the last 50 bytes of the file printed to the
console.

```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int fd1, fd2;
    char buf[1024];
    int n;

    // check arguments
    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        exit(1);
    }

    // open source file
    fd1 = open(argv[1], O_RDONLY);
    if (fd1 < 0) {
        perror("Error opening source file");
        exit(1);
    }

    // create destination file
    fd2 = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd2 < 0) {
        perror("Error creating destination file");
        close(fd1);
        exit(1);
    }

    // copy data
    while ((n = read(fd1, buf, sizeof(buf))) > 0) {
        if (write(fd2, buf, n) != n) {
            perror("Write error");
            close(fd1);
            close(fd2);
            exit(1);
        }
    }

    if (n < 0) {
        perror("Read error");
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully\n");
    return 0;
}
//then cc <7b.c>
//./a.out file1.txt file2.txt

```

//you should see the file copied successfully.

1.

nano check.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main(int argc, char *argv[]) {  
    if (argc != 2) {  
        printf("Usage: %s filename\n", argv[0]);  
        return 1;  
    }  
  
    if (access(argv[1], F_OK) == 0)  
        printf("File exists\n");  
    else  
        printf("File does not exist\n");  
  
    if (access(argv[1], R_OK) == 0)  
        printf("Readable\n");  
  
    if (access(argv[1], W_OK) == 0)  
        printf("Writable\n");  
  
    if (access(argv[1], X_OK) == 0)  
        printf("Executable\n");  
  
    return 0;  
}
```

2.

```
cc check.c -o check
```

3.

nano parent.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {  
    int pid = fork();  
  
    if (pid == 0) {  
        // Child process  
        printf("Child PID: %d, Parent PID: %d\n", getpid(),  
getppid());  
  
        execl("./check", "check", "file.txt", NULL);  
    }  
    else {  
        // Parent process  
        printf("Parent PID: %d, Child PID: %d\n", getpid(), pid);  
  
        wait(NULL); // wait for child  
        printf("Child finished execution\n");  
    }  
}
```

```
    }  
    return 0;  
}  
4.  
cc parent.c -o parent  
  
//then cc <parent.c>  
//./parent  
//you should see the parent and child PIDs and the file exists,  
readable, writable, and executable.
```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_RDONLY);
    char buf[21];
    int n;

    if (fd < 0) {
        perror("open");
        return 1;
    }

    // i. Read first 20 characters
    n = read(fd, buf, 20);
    buf[n] = '\0';
    printf("First 20 chars: %s\n", buf);

    // ii. Seek to 10th byte from beginning
    lseek(fd, 10, SEEK_SET);
    n = read(fd, buf, 20);
    buf[n] = '\0';
    printf("From 10th byte: %s\n", buf);

    // iii. Seek 10 bytes ahead from current position
    lseek(fd, 10, SEEK_CUR);
    n = read(fd, buf, 20);
    buf[n] = '\0';
    printf("After skipping 10 bytes: %s\n", buf);

    // iv. Display file size
    off_t size = lseek(fd, 0, SEEK_END);
    printf("File size: %ld bytes\n", size);

    close(fd);
    return 0;
}
//then cc <9b.c>
//./a.out
//you should see the first 20 characters, the characters from the
10th byte, the characters after skipping 10 bytes, and the file
size.

```



```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;
    int status;

    // First child
    pid1 = fork();
    if (pid1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        return 10;
    }

    // Second child
    pid2 = fork();
    if (pid2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        return 20;
    }

    // Parent using wait()
    pid_t w = wait(&status);
    printf("Parent: wait() returned PID = %d, exit = %d\n",
        w, WEXITSTATUS(status));

    // Parent using waitpid()
    w = waitpid(pid1, &status, 0);
    printf("Parent: waitpid() for PID %d, exit = %d\n",
        w, WEXITSTATUS(status));

    return 0;
}
//then cc <10b.c>
//./a.out
//you should see the parent and child PIDs and the exit status of
the children.

```

1.

parent.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid = fork();

    if (pid == 0) {
        // child executes another program
        execl("./add", "add", "10", "20", NULL);
    } else {
        // parent waits
        wait(NULL);
        printf("Parent: Child completed\n");
    }

    return 0;
}
```

2.

add.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s num1 num2\n", argv[0]);
        return 1;
    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    printf("Sum = %d\n", a + b);

    return 0;
}
```

3.

```
cc add.c -o add
cc parent.c -o parent
./parent
```

1.

parent.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid = fork();

    if (pid == 0) {
        // Child: user-defined environment
        char *env[] = {"VAR1=Hello", "VAR2=World", NULL};
        execl("./echoall", "echoall", "Child Process", NULL, env);
    }
    else {
        wait(NULL);

        // Parent: default environment
        execl("./echoall", "echoall", "Parent Process", NULL);
    }

    return 0;
}
```

2.echoall.c

```
#include <stdio.h>

extern char **environ;

int main(int argc, char *argv[]) {
    printf("Arguments:\n");
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    printf("\nEnvironment Variables:\n");
    for (int i = 0; environ[i] != NULL; i++) {
        printf("%s\n", environ[i]);
    }

    return 0;
}
```

3.

```
cc echoall.c -o echoall
cc parent.c -o parent
./parent
```